

Pint-OS - Alarm Clock

1. Objetivo

Substituir a espera ocupada de ``timer_sleep()`` por uma implementacao baseada em bloqueio de threads e interrupcao do temporizador.

2. Implementacao

Cada thread possui o campo ``wakeup_tick``, que armazena o tick em que ela pode voltar a executar. ``timer_sleep()`` ignora duracoes menores ou iguais a zero, desabilita as interrupcoes para atualizar o estado compartilhado, insere a thread em uma lista ordenada e chama ``thread_block()``.

No handler ``timer_interrupt()``, o contador global de ticks e incrementado. As threads no inicio da lista cujo ``wakeup_tick`` ja venceu sao removidas e passadas para ``thread_unblock()``. Como o handler nao pode dormir nem adquirir locks, a coordenacao entre handler e threads e protegida com interrupcoes desabilitadas.

Essa estrategia elimina o busy wait de ``timer_sleep()``: a thread dormindo nao consome CPU e somente retorna para a fila de prontas quando o prazo vence.

3. Scheduler

Para atender ao teste de prioridade, a fila de threads prontas passou a ser mantida em ordem decrescente de prioridade. Quando uma thread de prioridade maior e desbloqueada durante uma interrupcao, e solicitada uma troca de thread no retorno da interrupcao.

4. Testes

Teste	Resultado
---	---
alarm-single	PASS
alarm-multiple	PASS
alarm-simultaneous	PASS
alarm-priority	PASS
alarm-zero	PASS
alarm-negative	PASS

Os testes foram compilados no diretorio ``pint-os/src/threads`` com ``gcc -m32``, ``ld`` e ``objcopy`` nativos, usando QEMU como simulador. O linker cross-compiled ``i386-elf-ld`` nao estava instalado no ambiente.

5. Dificuldades

A primeira execucao via QEMU encerrava o kernel, mas o utilitario ``pintos`` esperava o timeout porque o dispositivo de encerramento nao estava configurado. O utilitario foi ajustado para usar ``isa-debug-exit`` e reconhecer o codigo de saida normal correspondente ao desligamento do Pintos.

Tambem foi necessario ordenar a fila de prontas por prioridade para que ``alarm-priority`` executasse as threads acordadas na ordem esperada.

6. Conclusao

A implementacao foi compilada e os seis testes do alarm clock foram aprovados. O caminho principal usa bloqueio, lista ordenada e o handler do temporizador, sem espera ocupada em ``timer_sleep()``.